PHANTOM CHAT

Secure Ephemeral Messaging Application

PRODUCT REQUIREMENTS DOCUMENT

Version	1.0
Status	Draft
Date	25 March 2026
Classification	Confidential
Target Platform	Oracle Cloud VM — Ubuntu 22.04 LTS
Tech Stack	Python / FastAPI / Caddy / PGP

1. Product Overview

Phantom Chat is a zero-persistence, end-to-end encrypted messaging application built for maximum operational security. The system is designed on the principle that the server should learn as little as possible about communications: all message content is encrypted before it reaches the server, no chat history is retained, and all session state is ephemeral.

The application is deployed on an Oracle Cloud VM with Caddy acting as a TLS-terminating reverse proxy, forwarding WebSocket and HTTP traffic to a FastAPI backend. PGP encryption is performed exclusively on the client side; the server acts as a ciphertext relay only.

1.1 Guiding Principles

- Zero Knowledge — the server never processes plaintext message content
- Zero Persistence — no logs, no database, no chat history survives a session
- Minimal Attack Surface — fewest possible dependencies, no unnecessary endpoints
- Defence in Depth — multiple independent security layers
- Fail Secure — any failure closes the session rather than degrading security

2. System Architecture

2.1 High-Level Architecture

The system consists of three main layers:

- Client Layer — Browser-based frontend performing PGP key generation, encryption, and decryption entirely in-browser using OpenPGP.js
- Transport Layer — Caddy reverse proxy on Oracle Cloud handling TLS 1.3 termination, HTTP/2, and automatic certificate management via ACME/Let's Encrypt
- Application Layer — FastAPI backend managing WebSocket sessions, ephemeral key exchange, and ciphertext forwarding

2.2 Communication Flow

Step 1	Client A opens the app; browser generates an ephemeral PGP keypair (4096-bit RSA or Curve25519) in memory
Step 2	Client A creates a session via REST; receives a short-lived session token (UUID, 30-minute TTL)
Step 3	Client A connects via WSS and publishes its public key to the session room
Step 4	Client B joins the session and receives Client A's public key
Step 5	Both clients exchange public keys; private keys never leave the browser
Step 6	All messages are encrypted client-side with the recipient's public key before transmission
Step 7	Server receives, validates (structure only, not content), and forwards ciphertext
Step 8	Recipient decrypts locally with their private key
Step 9	On session end or disconnect, all server-side session state is purged immediately

2.3 Oracle Cloud / Caddy Configuration

- Single Oracle Cloud Always Free VM (Ubuntu 22.04 LTS, AMD, 1 OCPU, 1 GB RAM minimum)
- Caddy handles automatic TLS certificate issuance and renewal (ACME/Let's Encrypt)
- Caddy proxies HTTPS (port 443) and WSS to FastAPI (localhost:8000)
- Oracle Cloud Security List: only ports 80, 443, and 22 (restricted) open inbound
- UFW firewall mirrors Oracle Security List rules; denies all other traffic
- Caddy access logs stripped of IP addresses; no application-level request logging

3. Security Requirements

3.1 Encryption

SEC-01	PGP E2E	All message content MUST be PGP-encrypted client-side before transmission. The server MUST NOT receive plaintext.	Critical
SEC-02	Key Generation	Keypairs MUST be generated in-browser using OpenPGP.js. Private keys MUST never be transmitted or persisted.	Critical
SEC-03	Key Size	Minimum key size: 4096-bit RSA or Curve25519 (preferred for performance).	High
SEC-04	Transport TLS	All connections MUST use TLS 1.3. TLS 1.2 MUST be disabled. HTTP MUST redirect to HTTPS.	Critical
SEC-05	Cipher Suites	Caddy MUST be configured to use only AEAD cipher suites (AES-GCM, ChaCha20-Poly1305).	High

3.2 Session & Authentication

SEC-06	Session Tokens	Session tokens MUST be cryptographically random UUIDs (128-bit entropy). Stored server-side in memory only.	Critical
SEC-07	Token TTL	Session tokens MUST expire after 30 minutes of inactivity. Expired tokens MUST be purged immediately.	High
SEC-08	No Accounts	The system MUST NOT require user accounts, email addresses, or any persistent identity.	Critical
SEC-09	Rate Limiting	Session creation and WebSocket connection endpoints MUST be rate-limited per IP (10 req/min).	High
SEC-10	CSRF Protection	All REST endpoints MUST enforce CSRF protection via SameSite cookies and Origin header validation.	High

3.3 Data Minimisation & Logging

SEC-11	No Message Logging	The server MUST NOT log any message content (ciphertext or otherwise).	Critical
SEC-12	No IP Logging	Access logs MUST strip IP addresses. Only aggregate metrics (connection count) are permitted.	High
SEC-13	No Persistence	No database. All session state lives in application memory and is lost on restart.	Critical
SEC-14	Memory Purge	On WebSocket close, all in-memory session data for that connection MUST be deleted synchronously.	Critical

SEC-15	No Chat History	The server MUST NOT buffer, store, or replay messages. Clients joining mid-session see no prior messages.	Critical

3.4 Input Validation & Hardening

SEC-16	Payload Validation	All WebSocket frames MUST be validated against a Pydantic schema. Oversized payloads (>64 KB) MUST be rejected.	High
SEC-17	Content-Type Enforcement	REST endpoints MUST reject non-JSON Content-Type requests.	Medium
SEC-18	Security Headers	HTTP responses MUST include: CSP, HSTS (max-age=63072000), X-Frame-Options: DENY, X-Content-Type-Options: nosniff, Referrer-Policy: no-referrer.	High
SEC-19	Dependency Pinning	All Python and JS dependencies MUST be pinned to exact versions. Lock files committed to VCS.	High
SEC-20	SBOM	A Software Bill of Materials MUST be generated at build time.	Medium

4. Functional Requirements

4.1 Session Management

FR-01	Create Session	A user can create a new chat session and receive a shareable session code.	Must Have
FR-02	Join Session	A user can join an existing session using the session code.	Must Have
FR-03	Session Capacity	A session MUST support a configurable maximum of N participants (default: 2).	Must Have
FR-04	Session Expiry	Sessions expire after 30 minutes of inactivity or when all participants disconnect.	Must Have
FR-05	Session Deletion	Explicit session deletion endpoint; purges all in-memory state immediately.	Should Have

4.2 Messaging

FR-06	Real-time Messaging	Messages MUST be delivered in real-time via WebSocket (WSS).	Must Have
FR-07	Message Delivery	Delivery confirmation receipts (ACK frames) MUST be sent to the sender.	Should Have
FR-08	Message Format	Messages MUST be structured JSON: {session_id, sender_fingerprint, ciphertext, timestamp}.	Must Have
FR-09	Typing Indicators	Optional ephemeral typing indicator events (no storage).	Could Have
FR-10	Participant Status	Online/offline status broadcast within session scope only.	Should Have

4.3 Key Exchange

FR-11	Public Key Publication	On WebSocket connect, client MUST publish its armoured PGP public key.	Must Have
FR-12	Key Distribution	Server MUST relay public keys to all session participants.	Must Have
FR-13	Key Fingerprint Display	UI MUST display the key fingerprint for manual verification by users.	Must Have
FR-14	Key Validation	Server MUST reject malformed public key payloads (basic structure check only).	Must Have

5. Non-Functional Requirements

5.1 Performance

- WebSocket message round-trip latency < 100ms under normal load
- Support minimum 50 concurrent sessions on Oracle Always Free tier
- Server startup time < 5 seconds

5.2 Reliability

- Graceful WebSocket reconnection with session continuity (within TTL)
- Application MUST be managed by systemd with automatic restart on failure
- Health check endpoint: GET /health returns 200 OK

5.3 Maintainability

- 100% type-annotated Python (mypy strict mode)
- Test coverage > 80% (pytest)

- All code linted with ruff; formatted with black
- Pre-commit hooks enforcing lint, format, and secret scanning (detect-secrets)

6. Technical Stack

Language	Python 3.12+
Web Framework	FastAPI 0.111+ with uvicorn (ASGI)
WebSockets	FastAPI native WebSocket (via Starlette)
Data Validation	Pydantic v2
PGP (Server-side checks)	python-gnupg (structural validation only)
PGP (Client-side E2E)	OpenPGP.js 5.x
Reverse Proxy	Caddy v2 (automatic HTTPS)
Infrastructure	Oracle Cloud Always Free VM — Ubuntu 22.04 LTS
Process Manager	systemd
Containerisation	Docker + docker-compose (local dev / optional prod)
Testing	pytest, pytest-asyncio, httpx
Linting / Format	ruff, black, mypy (strict)
Secret Scanning	detect-secrets (pre-commit)
Dependency Audit	pip-audit

7. Project Structure

```
phantom-chat/ ├── .kiro/ │   └── steering/ │       ├── product.md │       ├── tech.md
│       ├── structure.md │       └── security.md ├── app/ │   ├── main.py
# FastAPI app factory │   ├── config.py              # Settings (pydantic-settings) │
├── routes/ │   │   ├── session.py         # Session create/delete │   │   ├──
health.py          # /health endpoint │   │   └── ws.py              # WebSocket
handler │   ├── models/ │   │   ├── session.py         # Pydantic session models │   │
└── message.py         # Pydantic message models │   ├── services/ │   │   ├──
session_store.py   # In-memory session registry │   │   ├── pgp_validator.py   #
Public key structure validation │   │   └── rate_limiter.py    # Sliding-window rate
limiter │   └── middleware/ │       ├── security_headers.py │       └── csrf.py ├──
frontend/ │   ├── index.html │   ├── app.js                # OpenPGP.js key gen +
chat logic │   └── style.css ├── infra/ │   ├── Caddyfile │   ├── phantom-chat.service
# systemd unit file │   └── docker-compose.yml ├── tests/ │   ├── test_session.py │
├── test_ws.py │   └── test_security.py ├── .pre-commit-config.yaml ├── pyproject.toml
├── requirements.txt ├── requirements-dev.txt └── README.md
```

8. Threat Model (STRIDE Summary)

T-01	Spoofing	Ephemeral session tokens; manual fingerprint verification displayed in UI.	Mitigated
T-02	Tampering	TLS in transit; PGP signature verification on client.	Mitigated
T-03	Repudiation	No message logs; non-repudiation is intentionally out of scope.	Accepted
T-04	Info Disclosure	Server never sees plaintext; no logs; ephemeral memory only.	Mitigated
T-05	DoS	Rate limiting; max payload size; max session count; Caddy connection limits.	Partially Mitigated
T-06	Elevation of Privilege	No auth system; no persistent roles; read-only static frontend.	Mitigated
T-07	Traffic Analysis	Recommend Tor or VPN usage for metadata hiding (out of scope for MVP).	Accepted / Noted

9. Acceptance Criteria

9.1 Security Acceptance Tests

- SAT-01: Server-side message interception test — sniffing the WebSocket stream yields only PGP ciphertext
- SAT-02: Memory inspection post-session — no message content recoverable after disconnect
- SAT-03: TLS audit via testssl.sh — A+ rating
- SAT-04: OWASP ZAP passive scan — zero high-severity findings
- SAT-05: pip-audit — zero known CVEs in production dependencies

9.2 Functional Acceptance Tests

- FAT-01: Two clients can exchange PGP-encrypted messages end-to-end
- FAT-02: Session expires and purges correctly after TTL
- FAT-03: Oversized payload (>64 KB) is rejected with HTTP 413
- FAT-04: Rate limit triggers HTTP 429 after 10 requests/min
- FAT-05: Health endpoint returns 200 OK within 500ms

10. Development Milestones

M1 — Foundation	FastAPI skeleton, Pydantic models, health endpoint, basic tests, CI pipeline

M2 — Session Layer	In-memory session store, create/delete endpoints, rate limiter, CSRF middleware
M3 — WebSocket Core	WSS handler, key exchange relay, ciphertext forwarding, session TTL purge
M4 — Frontend	HTML/JS client with OpenPGP.js integration, key gen, encrypt/decrypt, fingerprint display
M5 — Hardening	Security headers middleware, payload validation, pip-audit gate, pre-commit hooks
M6 — Infrastructure	Caddyfile, systemd unit, Oracle Cloud firewall config, TLS audit pass
M7 — QA & Release	Full acceptance test suite pass, ZAP scan clean, README complete

11. Out of Scope (MVP)

- User accounts, authentication, or identity management
- Multi-device session sync
- File or media transfer
- Group chats beyond configurable participant limit
- Mobile native applications
- Message delivery guarantees beyond best-effort WebSocket
- Traffic analysis / metadata protection (Tor integration)
- Server-side PGP key signing or Web of Trust

END OF DOCUMENT